

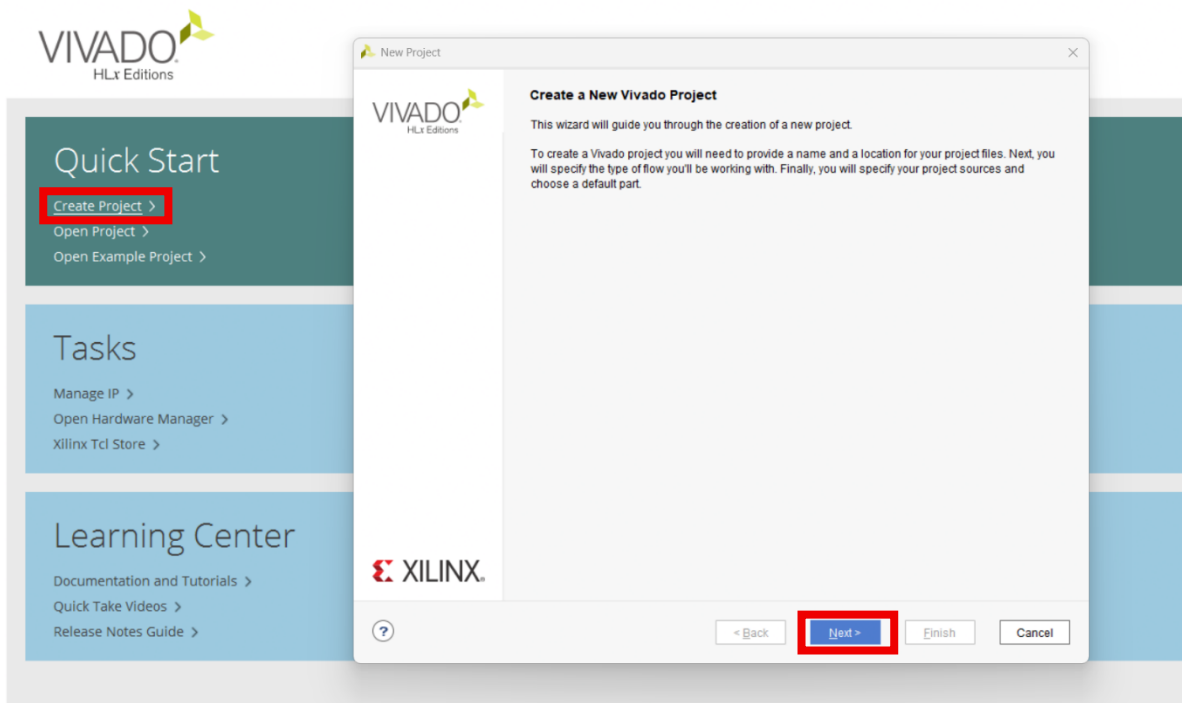
Lab 2

Objective of this lab

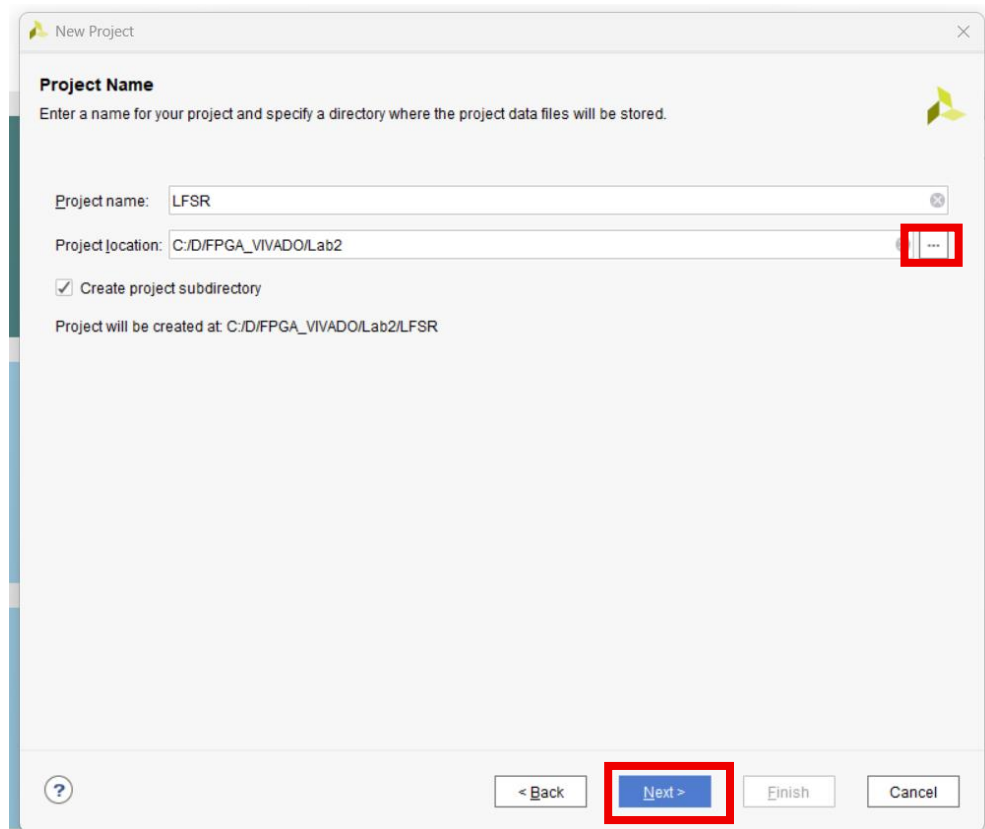
- This Lab implements and verifies a 4 MSB of LFSR on the Puzhi ZU4EV FPGA (xczu4ev-sfvc784-2-i) using debug IP cores:
 - **Virtual I/O (VIO):** Provides a JTAG-based GUI interface for runtime control (Enable, OUT_Enable), compensating for the board's lack of physical toggle switches.
 - **Integrated Logic Analyzer (ILA):** Monitors full-speed 25 MHz internal register transitions directly within Vivado without requiring external hardware probes.

Steps: -

1. You will find LFSR RTL in folder rtl and I/O pin mapping in folder xdc
2. Launch Vivado and create a new project inside a folder containing the 2 folders mentioned above



3. Specify the project's name and its location (click on the 3 points to specify the location)



New Project

Enter a name for your project and specify a directory where the project data files will be stored.

Project name: LFSR

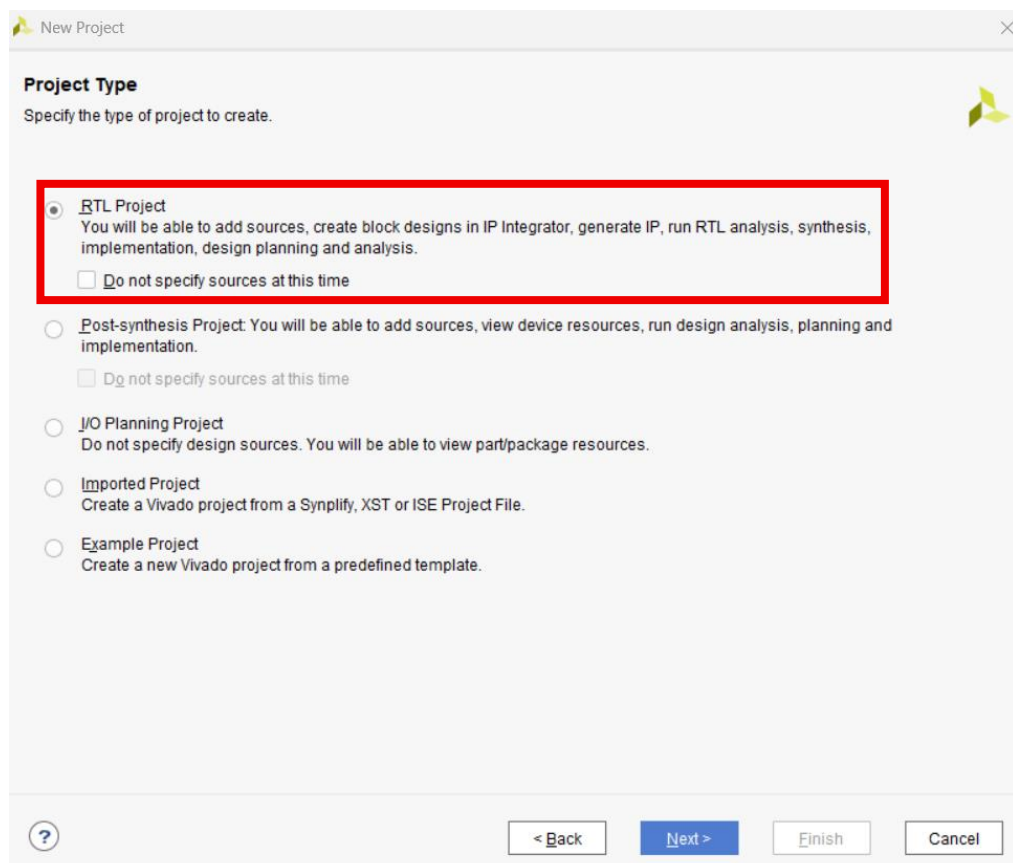
Project location: C:/FPGA_VIVADO/Lab2

☒ Create project subdirectory

Project will be created at: C:/FPGA_VIVADO/Lab2/LFSR

< Back Next > Finish Cancel

4. Select **RTL project** as the project type



New Project

Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.

☐ Do not specify sources at this time

☐ Post-synthesis Project: You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

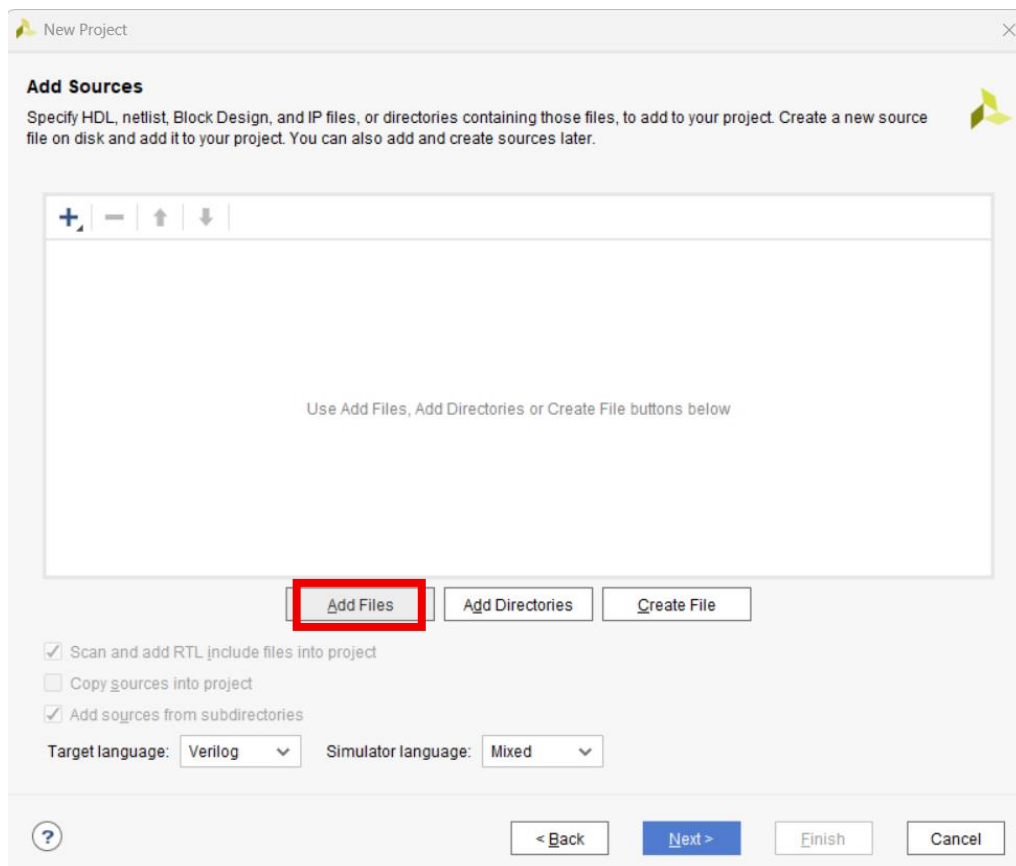
☐ I/O Planning Project
Do not specify design sources. You will be able to view part/package resources.

☐ Imported Project
Create a Vivado project from a Synplify, XST or ISE Project File.

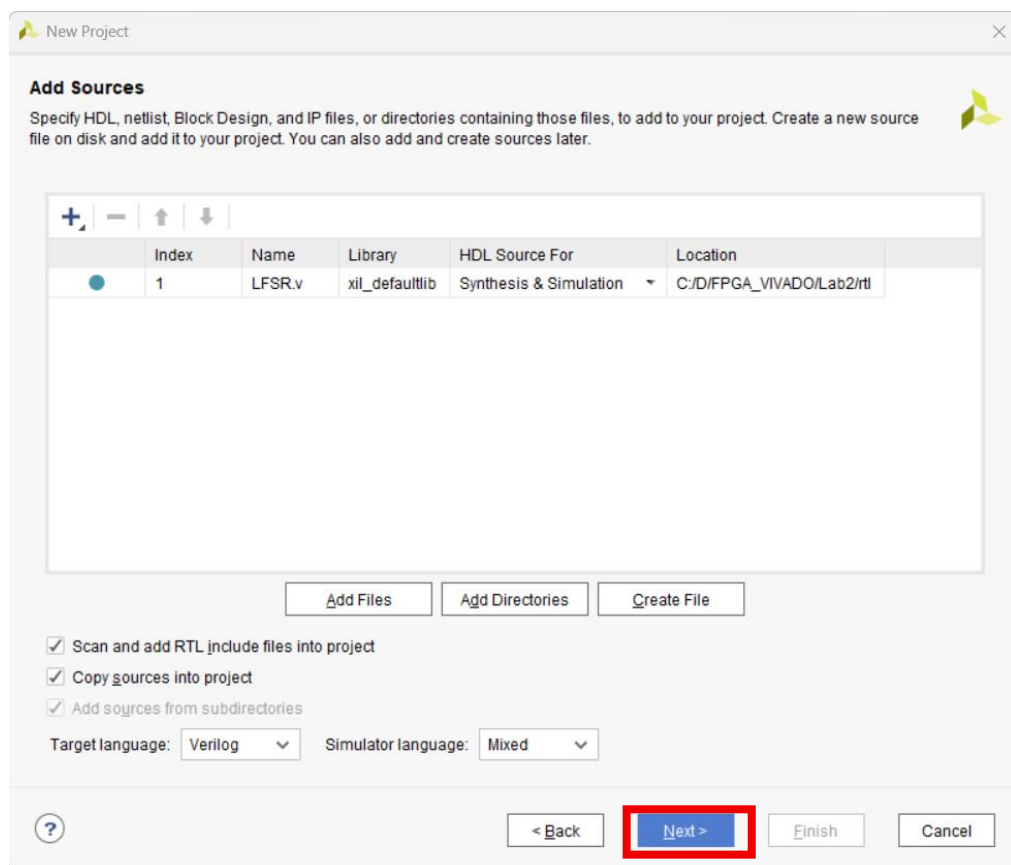
☐ Example Project
Create a new Vivado project from a predefined template.

< Back Next > Finish Cancel

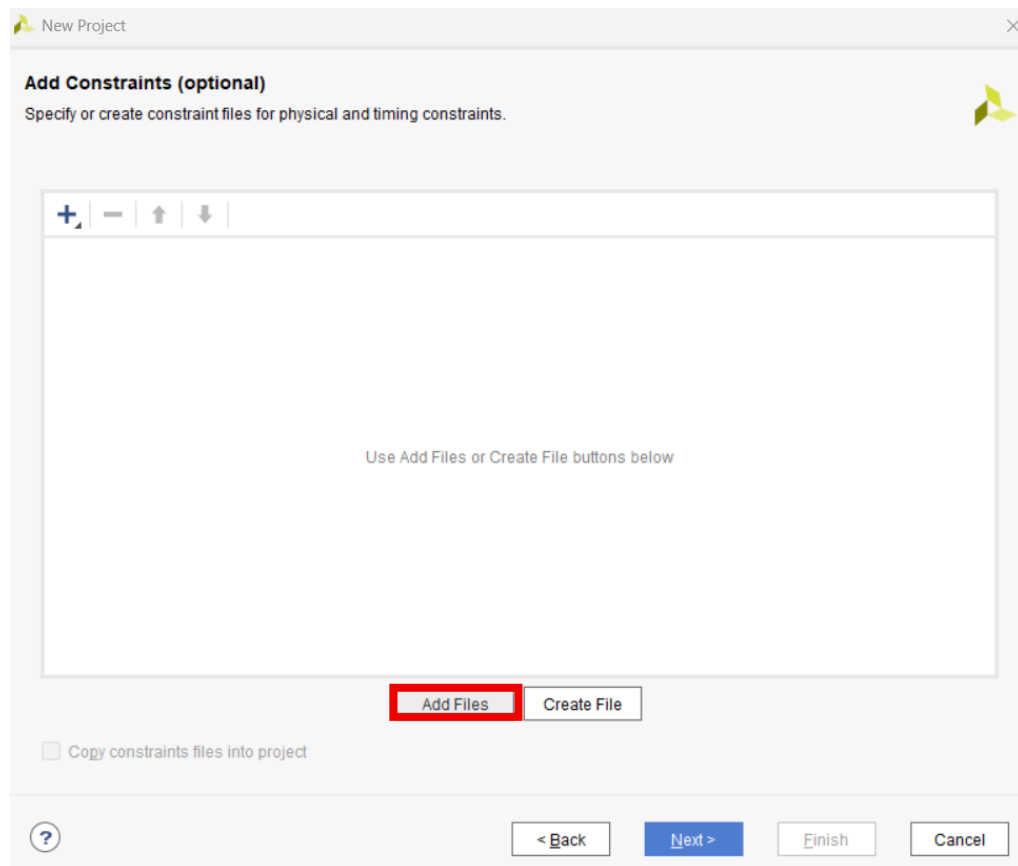
5. Add your RTL source files (click on Add Files)



- After adding your RTL files, it will be shown as below:



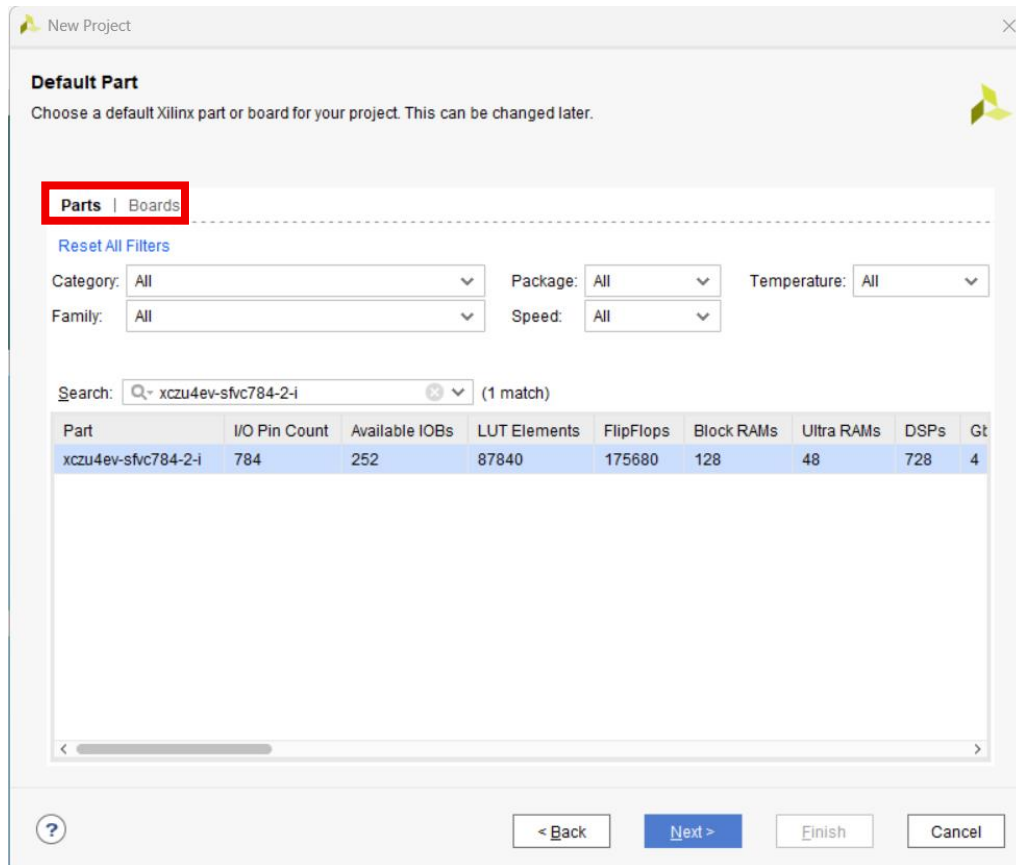
6. Add your **constraints (XDC)** file (click on Add Files)



- Your constraints file will have the input/output pin mapping and clock constraints. So, it will be like the following Pic:

```
1 # =====
2 # Constraints for design_1_wrapper (Puzhi ZU4EV Board)
3 # =====
4
5 # --- 200 MHz Differential Reference Clock (Bank 65) ---
6 set_property -dict {PACKAGE_PIN L3 IOSTANDARD DIFF_SSTL12} [get_ports {clk_200_p[0]}]
7 set_property -dict {PACKAGE_PIN L2 IOSTANDARD DIFF_SSTL12} [get_ports {clk_200_n[0]}]
8
9 create_clock -period 5.000 -name sys_clk_pin [get_ports {clk_200_p[0]}]
10
11 # --- Hardware Outputs (Carrier Board LEDs 1 & 2 - Bank 43, 3.3V) ---
12 set_property -dict {PACKAGE_PIN AD10 IOSTANDARD LVCMOS33} [get_ports {LFSR_OUT_0[3]}]
13 set_property -dict {PACKAGE_PIN AD11 IOSTANDARD LVCMOS33} [get_ports {LFSR_OUT_0[2]}]
14 set_property -dict {PACKAGE_PIN AF12 IOSTANDARD LVCMOS33} [get_ports {LFSR_OUT_0[1]}]
15 set_property -dict {PACKAGE_PIN AB10 IOSTANDARD LVCMOS33} [get_ports {LFSR_OUT_0[0]}]
16
17 # --- Hardware Reset Input (Carrier Board Key 2 - Bank 43, 3.3V) ---
18 set_property -dict {PACKAGE_PIN AH10 IOSTANDARD LVCMOS33} [get_ports Reset_0]
19
20 # --- Bitstream Compression ---
21 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
```

7. Select the target FPGA board (xczu4ev-sfvc784-2-i)



New Project

Default Part
Choose a default Xilinx part or board for your project. This can be changed later.

Parts | Boards

[Reset All Filters](#)

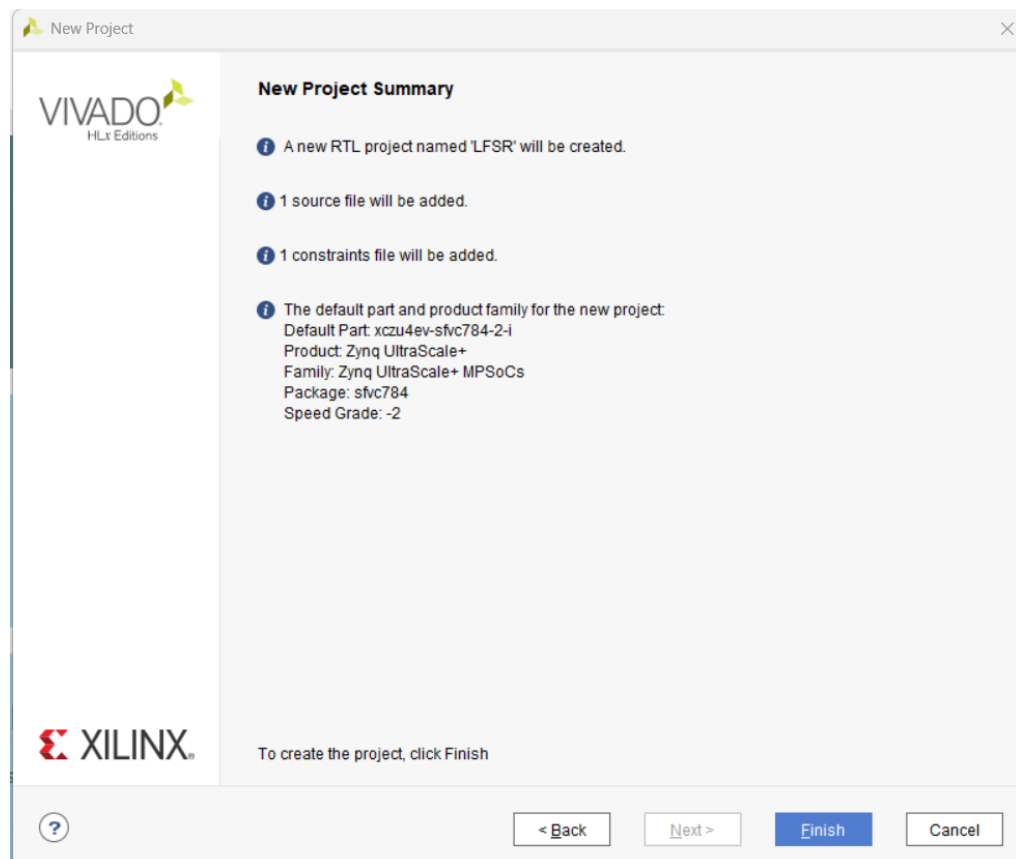
Category: All Package: All Temperature: All
Family: All Speed: All

Search: (1 match)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs	Gt
xczu4ev-sfvc784-2-i	784	252	87840	175680	128	48	728	4

[? Back](#) [Next >](#) [Finish](#) [Cancel](#)

8. Review of the selected project settings (click finish after your confirmation)



New Project

VIVADO
HLx Editions

New Project Summary

- i** A new RTL project named 'LFSR' will be created.
- i** 1 source file will be added.
- i** 1 constraints file will be added.
- i** The default part and product family for the new project:
Default Part: xczu4ev-sfvc784-2-i
Product: Zynq UltraScale+
Family: Zynq UltraScale+ MPSoCs
Package: sfvc784
Speed Grade: -2

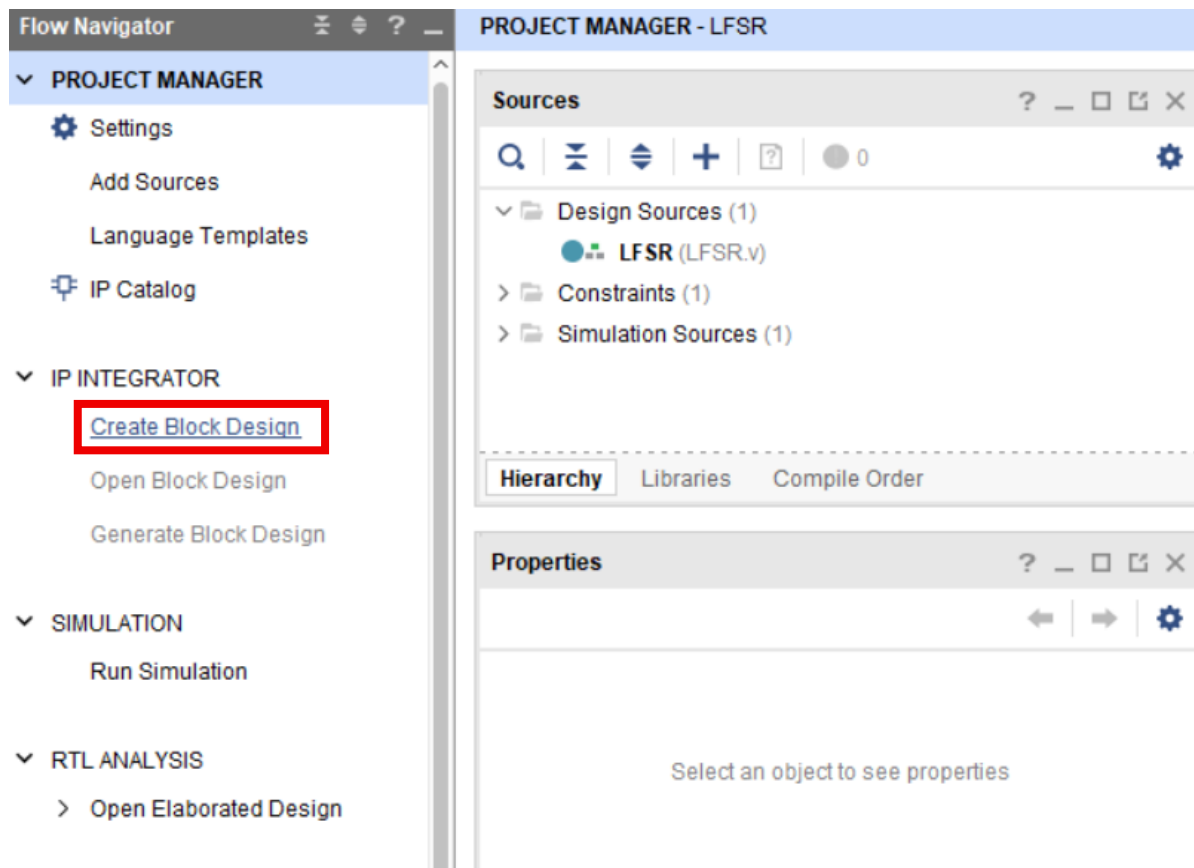
XILINX

To create the project, click Finish

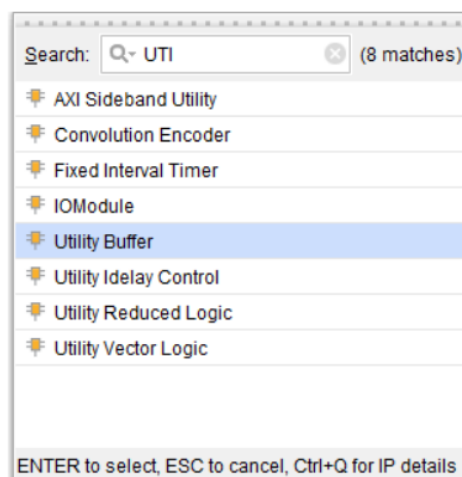
[? Back](#) [Next >](#) [Finish](#) [Cancel](#)

9. From IP INTEGRATOR:

1) create block design

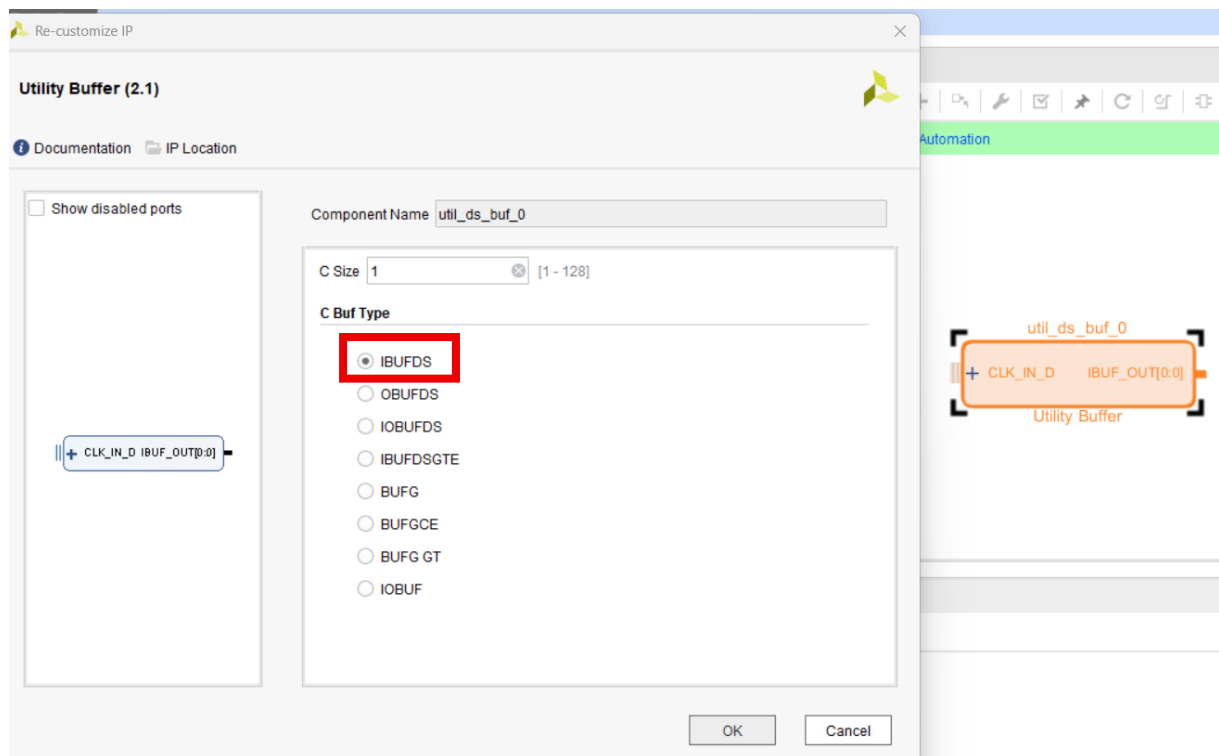


2) Click Add IP then search for Utility Buffer (we will use it to convert the differential clock output from the oscillator to single ended one)



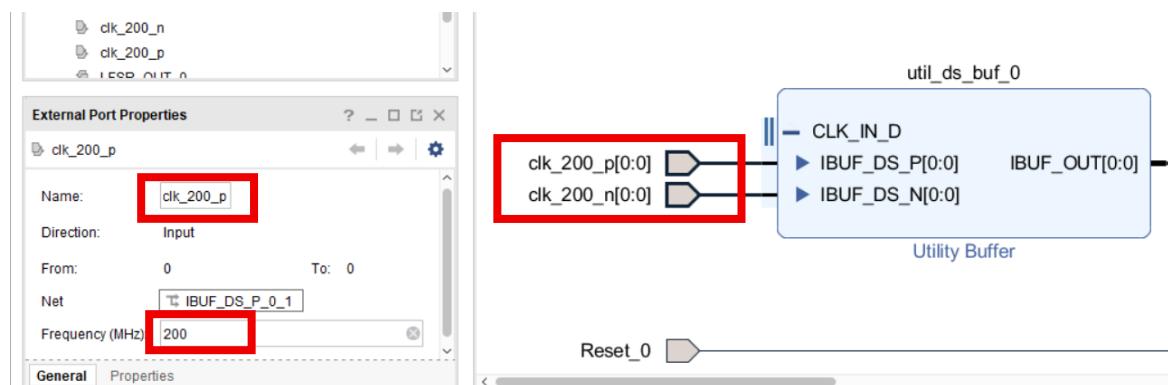
is empty. Press the + butt

3) Double Click on the buffer and choose IBUFDS

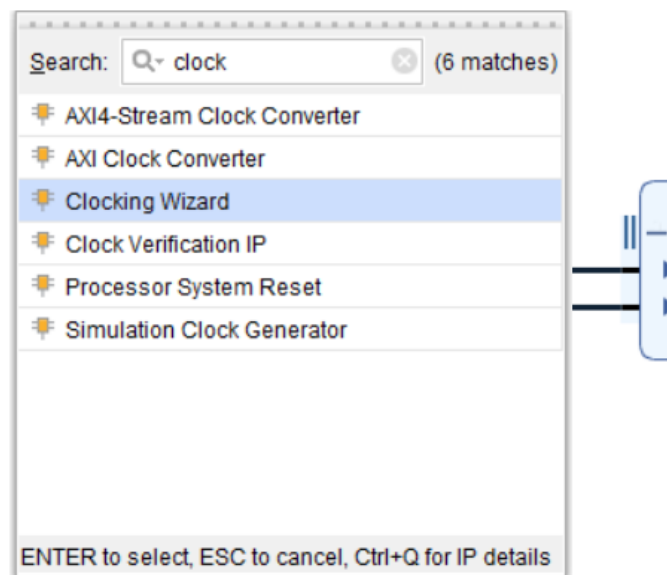


4) Right click on each input port of it then select **make external**

5) Click on input ports and rename its 2 clock input pins to **clk_200_p** and **clk_200_n** and change the frequency to 200 MHz



- 6) Add another IP by Clicking Add IP then search for Clock wizard (we will use it to generate 25 MHz from the 200 MHz)



- 7) Configure the clock to take 200 MHz and output 25 MHz and remove reset and locked ports

A screenshot of the Clocking Wizard configuration window. The window is divided into two main sections: 'IP Symbol' on the left and 'Resource' on the right. The 'IP Symbol' section shows a block diagram with ports 'reset', 'clk_in1', 'clk_out1', and 'locked'. The 'Resource' section contains the configuration options for the 'clk_wiz_0' component. The 'Clocking Options' tab is selected, showing 'MMCM' as the chosen option. The 'Clocking Features' section includes checkboxes for 'Frequency Synthesis' (checked), 'Minimize Power', 'Phase Alignment', 'Spread Spectrum', 'Dynamic Reconfig', 'Dynamic Phase Shift', 'Safe Clock Startup', and 'Use CDDC'. The 'Dynamic Reconfig Interface Options' section shows 'AXI4Lite' as the selected interface. The 'Input Clock Information' table is shown below, with the 'Primary' input clock 'clk_in1' set to a frequency of 200 MHz. The 'Secondary' input clock 'clk_in2' is set to 100.000 MHz. The '200' value in the 'Primary' row is highlighted with a red box.

Input Clock	Port Name	Input Frequency(MHz)
☒ Primary	clk_in1	200
☐ Secondary	clk_in2	100.000

IP Symbol

Resource

☐ Show disabled ports

Component Name

clk_wiz_0

Clocking Options

Output Clocks

MMCM Settings

The phase is calculated relative to clk_out1.

Output Clock	Port Name	Output Freq (MHz) Requested
☒ clk_out1	clk_out1	25
☐ clk_out2	clk_out2	100.000
☐ clk_out3	clk_out3	100.000
☐ clk_out4	clk_out4	100.000
☐ clk_out5	clk_out5	100.000
☐ clk_out6	clk_out6	100.000
☐ clk_out7	clk_out7	100.000

☐ USE CLOCK SEQUENCING

clk_in1

clk_out1

clk_in1

clk_out1

clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Enable Optional Inputs / Outputs for MMCM/PLL

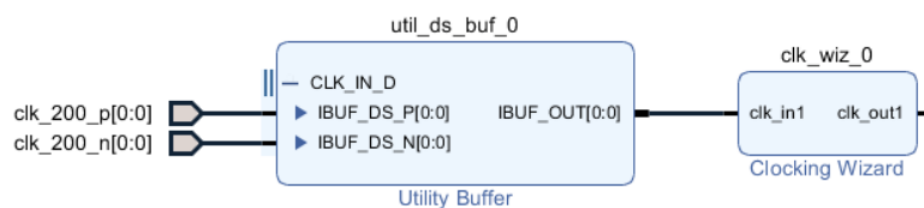
☐ reset
☐ power_down
☐ input_clk_stopped
☐ locked
☐ clkfbstopped

Reset Type

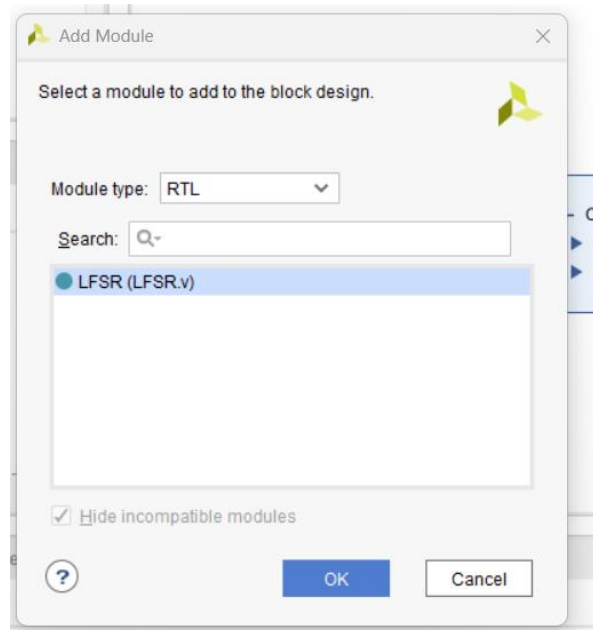
☒ Active High
☐ Active Low
☐ W

Phase Shift

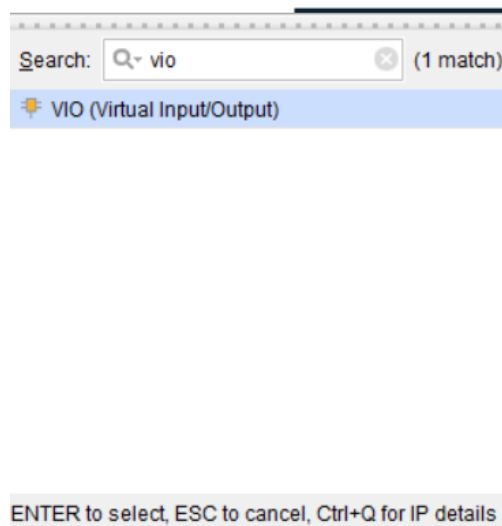
8) Connect the output from the utility buffer to the input of the clock wizard

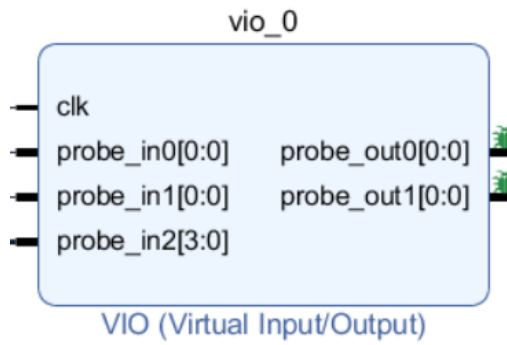


9) Now add your module by right clicking on empty space then select add module

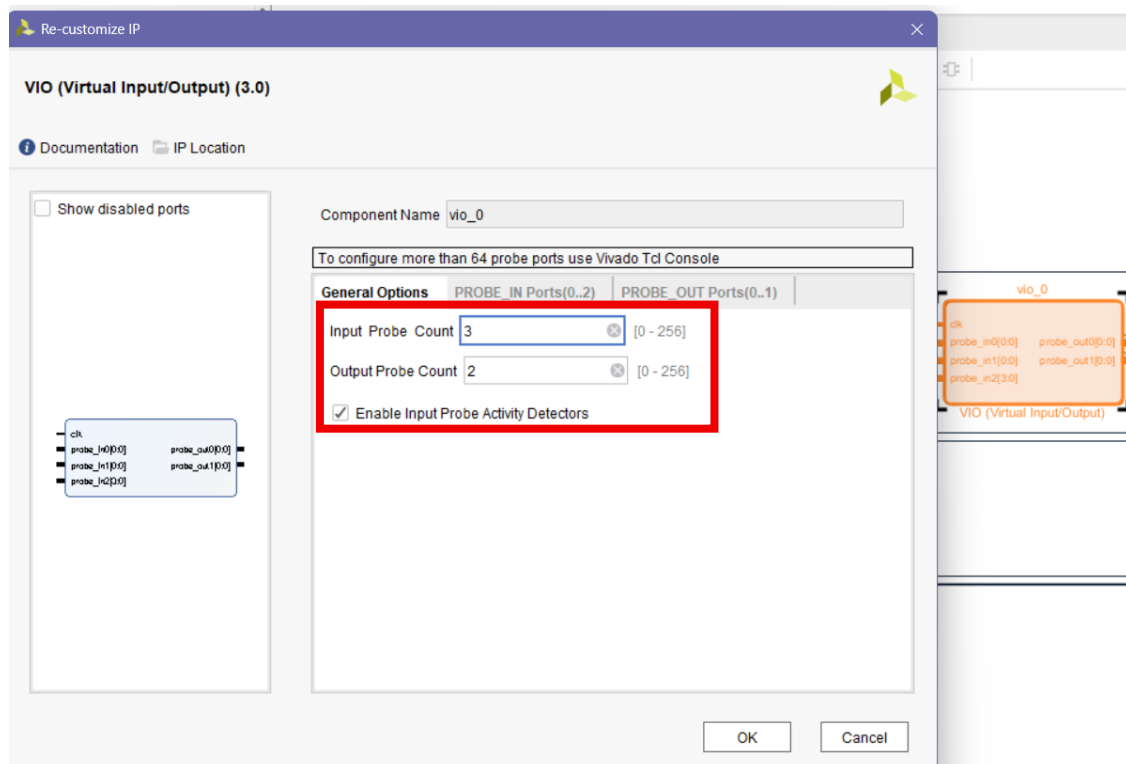


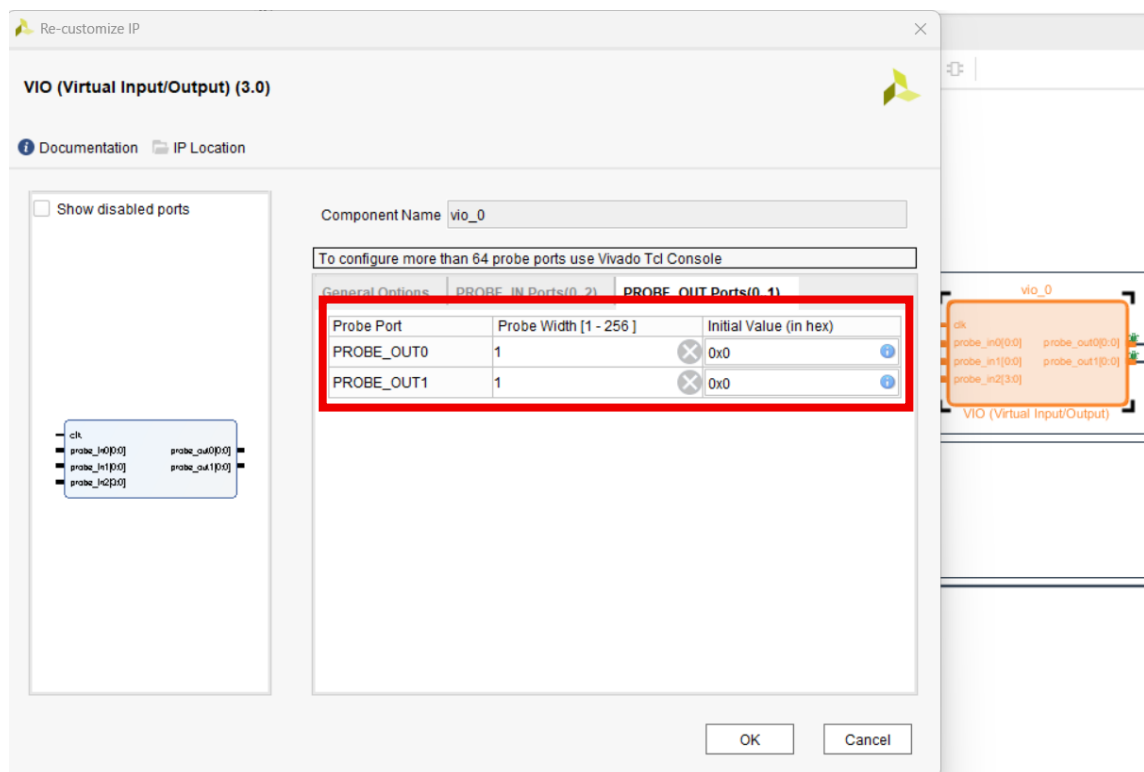
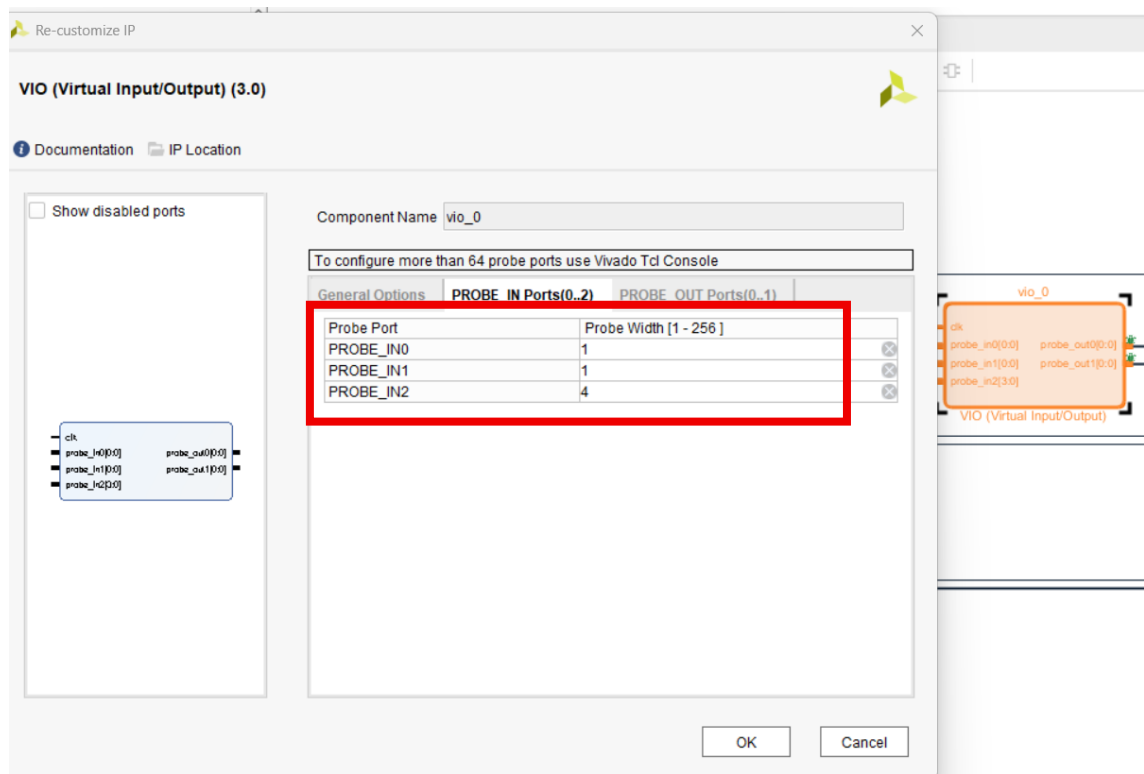
10) Add another IP by Clicking Add IP then search for VIO (Virtual Input / Output)





- 11) Double click on the VIO and do the following:
 - i. Make input probe count = **3** (from general options)
 - ii. Make output probe count = **2** (from general options)
 - iii. Change PROBE_IN2 width to **4** (from PROBE_IN_PORTS)
 - iv. The remaining ports leave it as 1 bit

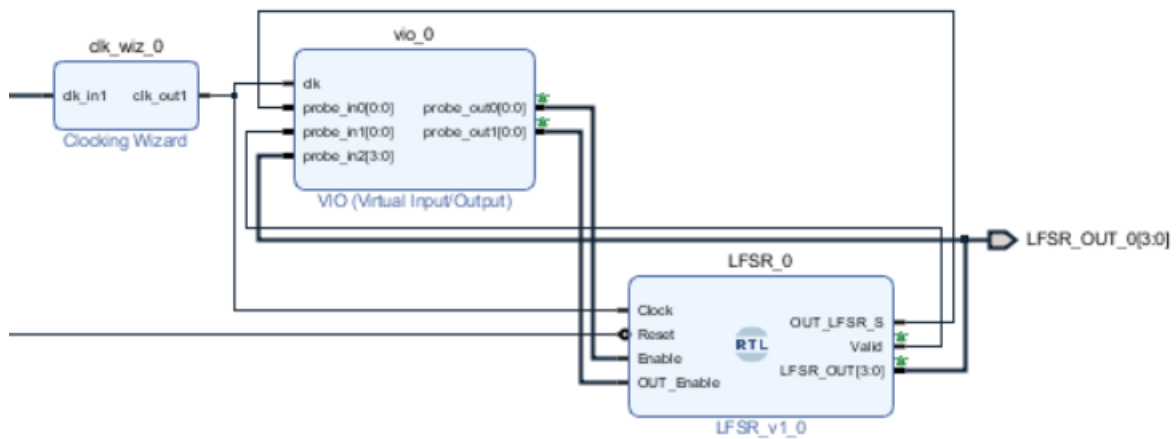




12) Do the following:

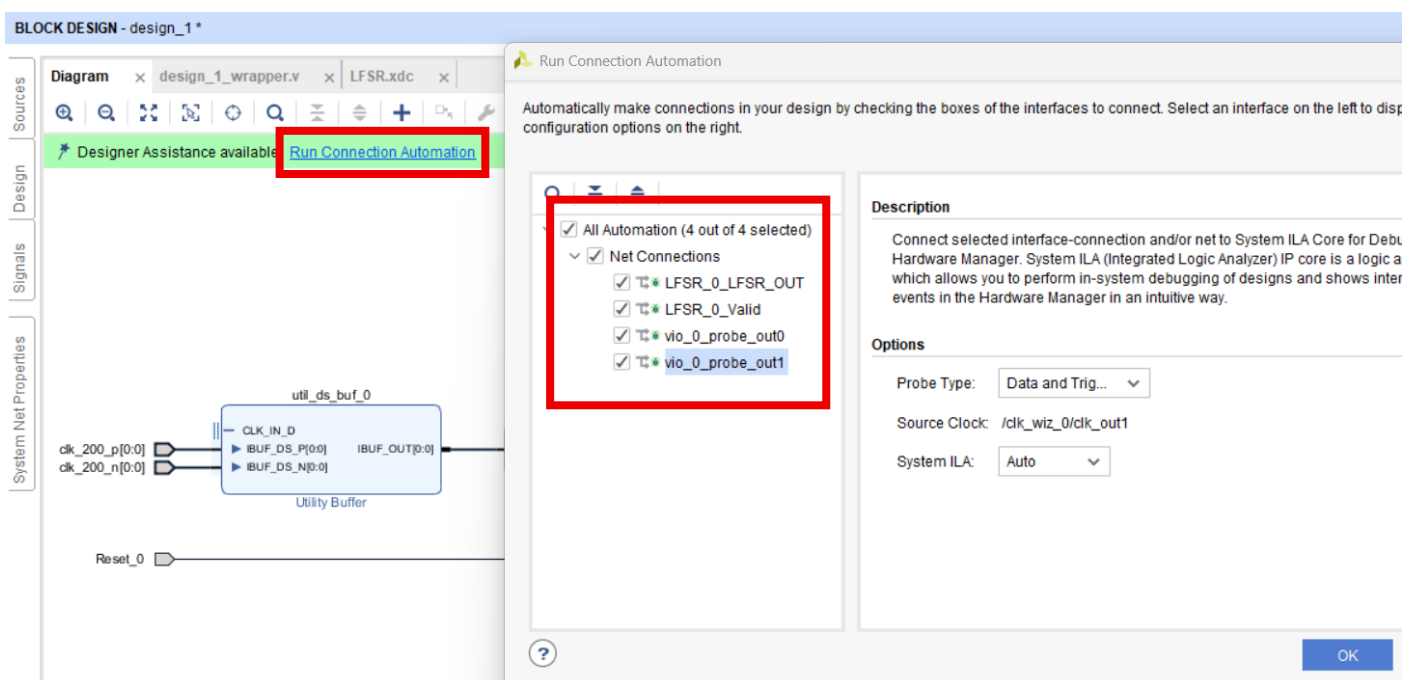
- i. Right click on LFSR_OUT on our module and select **make external**
- ii. Right click on Reset on our module and select **make external**
- iii. Connect **clock of VIO to clock of our module to clock output from clock wizard**
- iv. Connect Enable and OUT_Enable to the output ports from the VIO
- v. Connect OUT_LFSR_S and Valid to input ports of the VIO with 1 bit in width

vi. Connect LFSR_OUT to the input port of the VIO with 4 bits in width

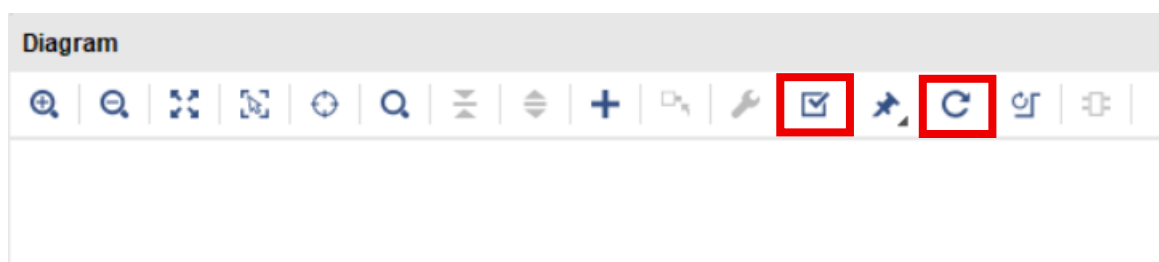


13) Right click on Enable, OUT_Enable, Valid and LFSR_OUT and select debug (Don't do anything to OUT_LFSR_S)

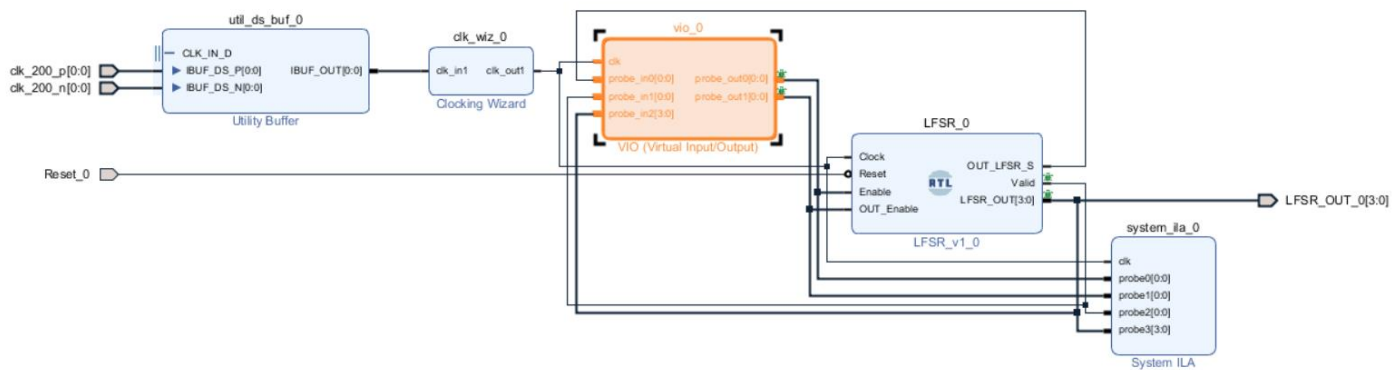
14) Run connection Automation then select all Net Connections



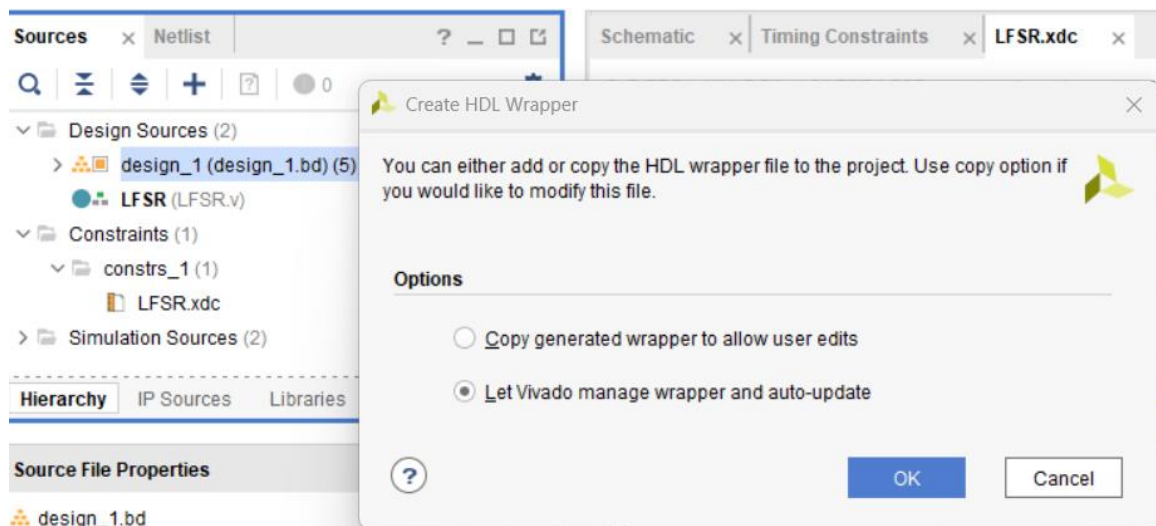
15) Click on regenerate layout then Validate design and solve any problem (clock of ILA will be connected on same clock of VIO and module clock)



16) This is supposed to be your final layout and connections



10. Right click on .bd file and select create HDL Wrapper



11. Open the Wrapper file and modify the constraint file with the right ports naming as it might change

12.Run Elaboration

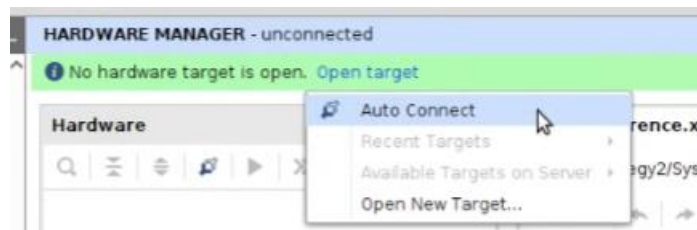
13.Run Synthesis (If there is a hold violation, it will be solved after implementation)

14.Run Implementation and check on timing report

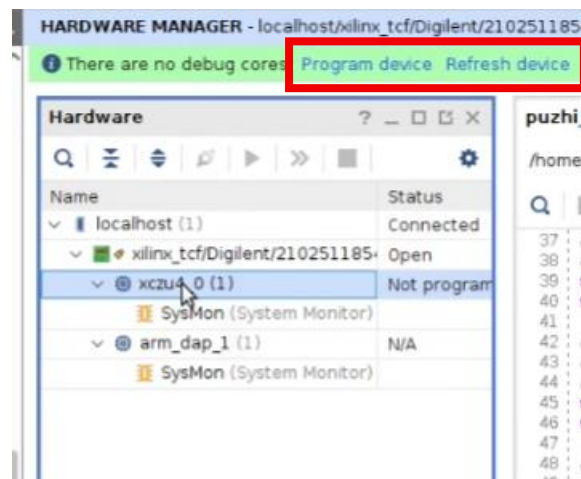
15.Generate Bitstream File

- After **Generate Bitstream** in Vivado, the flow is:

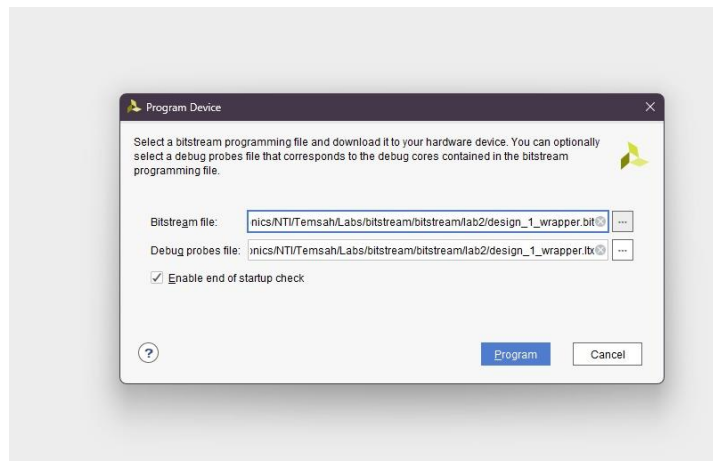
- Open Hardware Manager
- Open Target then Auto Connect



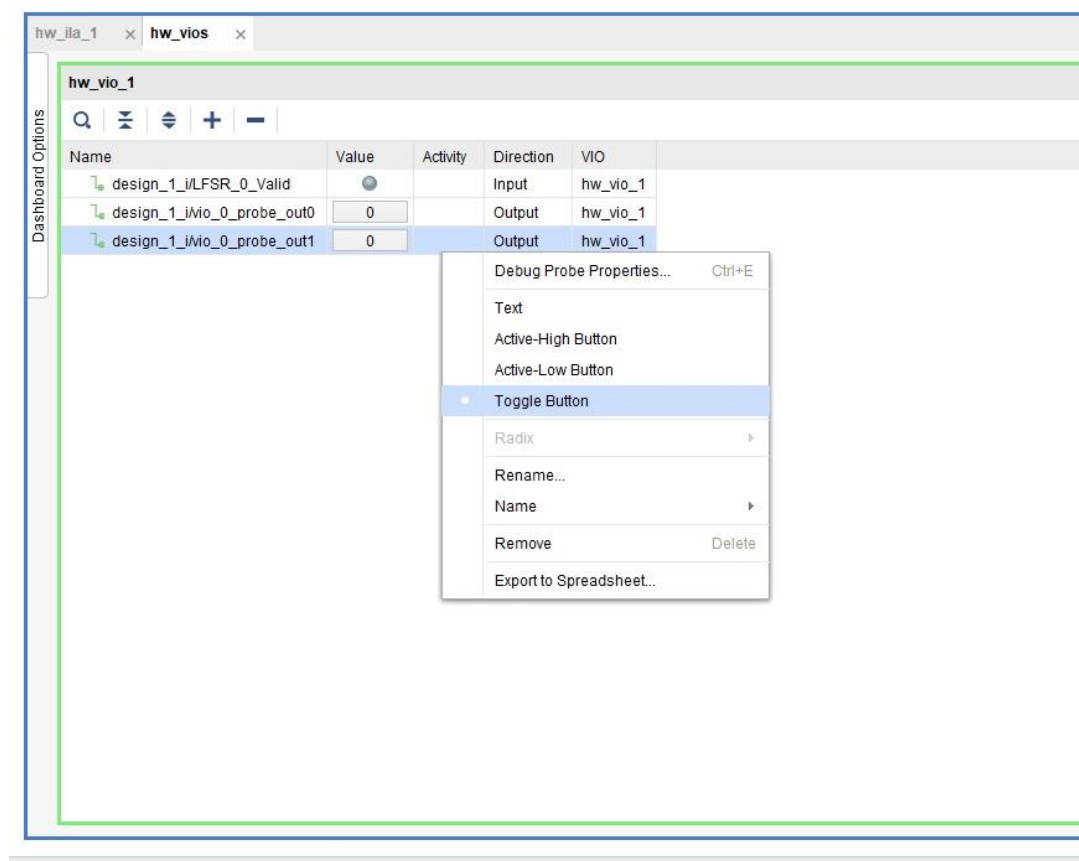
- Vivado should detect your FPGA board through JTAG.
- Program the FPGA
- Click: **Program Device**.

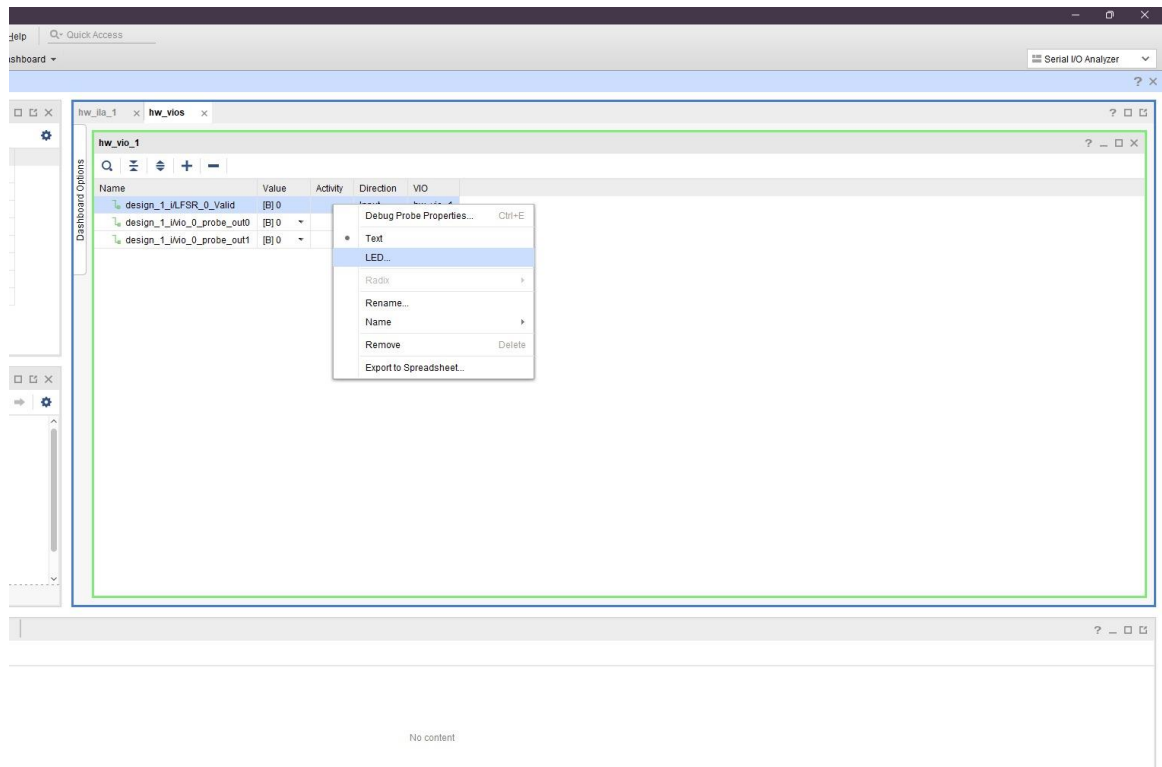


- Select your generated .bit file. Vivado normally finds it automatically.
- Then click **Program**. Your FPGA is now configured with your design



- You can control the input signals using VIO





- ILA will show debug signals as a waveform to trace

